



Algebra Relazionale

F. Proietto Salanitri



Linguaggi per le basi di dati

- ▶ Operazioni sullo schema
 - ▶ DDL: data definition language
- ▶ Operazioni sui dati
 - ▶ DML: data manipulation language
 - ▶ interrogazione ("query")
 - ▶ aggiornamento



Algebra Relazionale

Costituita da un insieme di operatori

- ▶ definiti su relazioni
- ▶ che producono relazioni
- ▶ e possono essere composti



Algebra Relazionale

Operatori dell'algebra relazionale:

- ▶ unione, intersezione, differenza
- ▶ Ridenominazione
- ▶ Selezione
- ▶ Proiezione
- ▶ join (join naturale, prodotto cartesiano, theta-join)



Operatori insiemistici

- ▶ le relazioni sono insiemi
 - ▶ i risultati debbono essere relazioni
 - ▶ è possibile applicare **unione**, **intersezione**, **differenza** solo a relazioni definite sugli stessi attributi
- 

Unione

Laureati

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Quadri

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureati $\cup$ Quadri

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45
9297	Neri	33

DIFFERENZA

Laureati

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Quadri

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureati – Quadri

Matricola	Nome	Età
7274	Rossi	42

intersezione

Laureati

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Quadri

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureati $\cap$ Quadri

Matricola	Nome	Età
7432	Neri	54
9824	Verdi	45

Operatori derivati dagli insiemi

Unione

L'*unione* fra due relazioni r_1 e r_2 definite sullo stesso insieme X si indica con $\mathbf{r_1 \cup r_2}$ ed è una relazione su X contenente le tuple appartenenti a r_1 , a r_2 o ad entrambe.

Intersezione

L'*intersezione* fra due relazioni r_1 e r_2 definite sullo stesso insieme X si indica con $\mathbf{r_1 \cap r_2}$ ed è una relazione su X contenente le tuple appartenenti sia a r_1 che a r_2 .

Differenza

La *differenza* fra due relazioni r_1 e r_2 definite sullo stesso insieme X si indica con $\mathbf{r_1 - r_2}$ ed è una relazione su X contenente le tuple appartenenti a r_1 ma non a r_2 .

Unione Sensata ma Impossibile

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

Maternità

Madre	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

Paternità $\cup$ Maternità

??

RIDENOMINAZIONE

- Adeguare i nomi degli attributi
- operatore monadico
- "modifica lo schema" lasciando inalterata l'istanza dell'operando

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

REN_{Genitore ← Padre} (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

Ridenominazione

La *ridenominazione* consente di modificare il nome di un attributo per associarlo ad un altro in una operazione algebrica.

Ad es. rende compatibili due attributi con nome diverso quando ha senso fare l'unione fra le relazioni cui appartengono.

Si indica con

$\rho_{\text{nuovonome}} \leftarrow \text{vecchionome} \text{ (Relazione)}$

Ridenominazione

- Sia r una relazione definita sull'insieme degli attributi X e sia Y un altro insieme di attributi con la stessa cardinalità.
- Siano $A_1, A_2, \dots, A_k$ e $B_1, B_2, \dots, B_k$ rispettivamente un ordinamento per gli attributi in X ed un ordinamento per quelli in Y .
- La ridenominazione è definita:
 - $\rho_{B_1, B_2, \dots, B_k \leftarrow A_1, A_2, \dots, A_k}(r)$
- Contiene una tupla t' per ciascuna tupla t in r definita come:
 - t' è una tupla in Y e $t'[B_i] = t[A_i]$ per $i = 1, \dots, k$

Ridenominazione

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

ρ Genitore $\leftarrow$ Padre (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

Maternità

Madre	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

ρ Genitore $\leftarrow$ Madre (Maternità)

Genitore	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

Ridenominazione

ρ Genitore $\leftarrow$ Padre (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

ρ Genitore $\leftarrow$ Madre (Maternità)

Genitore	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

ρ Genitore $\leftarrow$ Padre (Paternità)
 $\cup$

ρ Genitore $\leftarrow$ Madre (Maternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco
Eva	Abele
Eva	Set
Sara	Isacco

Ridenominazione

Impiegati

Cognome	Ufficio	Stipendio
Rossi	Roma	55
Neri	Milano	64

Operai

Cognome	Fabbrica	Salario
Bruni	Monza	45
Verdi	Latina	55

ρ Sede, Retribuzione $\leftarrow$ Ufficio, Stipendio (Impiegati)
 $\cup$
 ρ Sede, Retribuzione $\leftarrow$ Fabbrica, Salario (Operai)

Cognome	Sede	Retribuzione
Rossi	Roma	55
Neri	Milano	64
Bruni	Monza	45
Verdi	Latina	55



Selezione e Proiezione

- ▶ Le operazioni di **selezione** e di **proiezione** si applicano ad una relazione e ne restituiscono una porzione.
 - ▶ Possono essere considerate **ortogonali** o complementari, in quanto una opera sulle righe e l'altra sulle colonne.
 - ▶ La *selezione* produce un insieme di tuple, definite su tutti gli attributi della relazione.
 - ▶ La *proiezione* produce un risultato definito su un insieme limitato di attributi, cui contribuiscono tutte le tuple.
- 



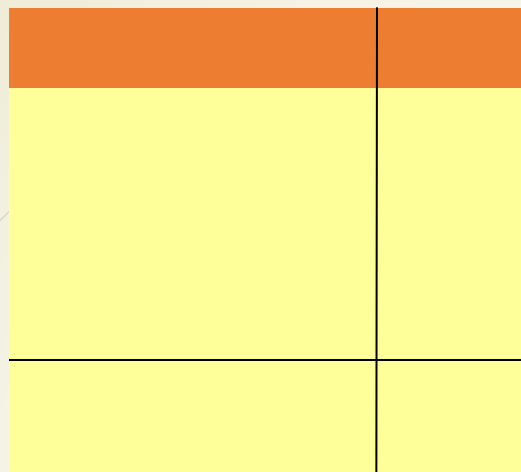
Selezione e Proiezione

- ▶ **selezione:**

- ▶ decomposizione orizzontale

- ▶ **proiezione:**

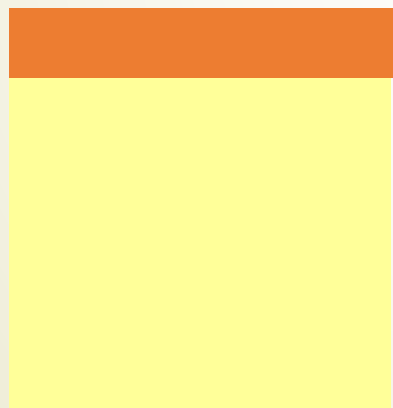
- ▶ decomposizione verticale



selezione



proiezione



Selezione

- ▶ La *selezione* produce una nuova relazione definita sugli stessi attributi della relazione cui è applicata, contenente solamente le tuple di quest'ultima che soddisfano una specifica *condizione di selezione*.
- ▶ Si indica con $\sigma_F(r)$ o $SEL_F(r)$ dove:
 - F è una condizione da verificare
 - r è la relazione a cui la selezione è applicata
- ▶ Quindi $\sigma_F(r)$ produce una relazione, sullo stesso schema



Selezione

Sintassi

σ *Condizione* (*Operando*)

- *Condizione*: espressione booleana (come quelle dei vincoli di ennupla)

Semantica

- il risultato contiene le ennuple dell'operando che soddisfano la condizione

Selezione

La *condizione di selezione* F è una *formula proposizionale* su X , cioè una formula ottenuta combinando con i simboli $\wedge$ (*and*) $\vee$ (*or*) $\neg$ (*not*) espressioni del tipo

$$A \theta B \quad \text{o} \quad A \theta c$$

- θ è un operatore di confronto ($\leq, <, =, >, \geq$)
- A e B sono attributi di X su cui il confronto abbia senso
- c è una costante tale che il confronto con A sia definito



Selezione

Definizione

La *selezione* $\sigma_F(r)$ produce una relazione sugli stessi attributi di r che contiene le tuple di r su cui F è vera.

Selezione

- ▶ impiegati che guadagnano più di 50

$SEL_{\text{Stipendio} > 50}$ (Impiegati)

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
5698	Neri	Napoli	64

$SEL_{\text{Stipendio} > 50}$ (Impiegati)

Selezione

- impiegati che guadagnano più di 50 e lavorano a Milano

SEL_{Stipendio > 50 AND Filiale = 'Milano'} (Impiegati)

Matricola	Cognome	Filiale	Stipendio
5998	Neri	Milano	64

SEL_{Stipendio > 50 AND Filiale = 'Milano'} (Impiegati)

Proiezione

Dati una relazione $r(X)$ e un sottoinsieme Y di X , la proiezione di r su Y si indica con

$$\Pi_Y (r) \quad \text{o} \quad \text{PROJ}_Y (r)$$

ed è l'insieme di tuple su Y ottenute dalle tuple di r considerando solo i valori su Y .

$$\Pi_Y (r) = \{ t[Y] \mid t \in r \}$$

Cardinalità delle proiezioni: Una proiezione ha un numero di tuple *minore* o *uguale* rispetto alla relazione r cui è applicata



Proiezione

Sintassi

$\text{PROJ}_{\text{ListaAttributi}} (\text{Operando})$

Semantica

Il risultato contiene le tuple che si ottengono restringendo tutte le tuple dell'operando agli attributi nella lista (ed eliminando gli eventuali duplicati)

Proiezione

Impiegati

Matricola	Cognome	Filiale	Stipendio
7309	Neri	Napoli	55
5998	Neri	Milano	64
9553	Rossi	Roma	44
5698	Rossi	Roma	64

- per tutti gli impiegati:
 - matricola e cognome
 - cognome e filiale



Proiezione

Matricola Cognome	
7309	Neri
5998	Neri
9553	Rossi
5698	Rossi

PROJ_{Matricola, Cognome} (Impiegati)

Proiezione

Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma

PROJ Cognome, Filiale (Impiegati)

Cardinalità delle proiezioni

- ▶ una proiezione
 - ▶ contiene al più tante ennuple quante l'operando
 - ▶ può contenerne di meno
- ▶ se X è una superchiave di R , allora $PROJ_X(R)$ contiene esattamente tante ennuple quante R



Join

- ▶ Combinando selezione e proiezione, possiamo estrarre informazioni da **una** relazione
- ▶ Non possiamo però correlare informazioni presenti in relazioni diverse, né informazioni in ennuple diverse di una stessa relazione
- ▶ Il **join** è l'operatore più interessante (potente) dell'algebra relazionale in quanto permette di correlare dati presenti in relazioni diverse
- ▶ Esistono due tipi principali di join:
 - ▶ Natural
 - ▶ Theta



Join NATURALE

- ▶ Operatore binario (generalizzabile)
 - ▶ L'operatore di join correla dati contenuti in relazioni diverse
 - ▶ Il suo risultato è una relazione definita sull'unione degli insiemi di attributi degli operandi, le cui tuple sono ottenute combinando le tuple degli operandi **che hanno valori uguali** su attributi comuni (con lo stesso nome).
- 

Join Naturale

- Il join naturale $r_1 \bowtie r_2$ di $r_1(X_1)$ e $r_2(X_2)$ è una relazione definita su $X_1 \cup X_2$ (che si può scrivere $X_1 X_2$) :

$$r_1 \bowtie r_2 = \{ t \text{ su } X_1 X_2 \mid t[X_1] \in r_1 \text{ e } t[X_2] \in r_2 \}$$

- Se $X_1 \cap X_2$ è vuoto il join naturale equivale al *prodotto cartesiano* fra le relazioni.
- Se $X_1 = X_2$ il join naturale equivale all'intersezione fra le relazioni

Join Naturale

Impiegato Reparto	
Rossi	A
Neri	B
Bianchi	B

Reparto	Capo
A	Mori
B	Bruni

Impiegato	Reparto	Capo
Rossi	A	Mori
Neri	B	Bruni
Bianchi	B	Bruni



Join Naturale

- ▶ Se ciascuna tupla di ciascun operando contribuisce ad almeno una tupla del risultato il join si dice *completo*.
- ▶ Se per alcune tuple non è verificata la corrispondenza e non contribuiscono al risultato, le tuple si dicono *dangling*.

Join

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparto	Capo
B	Mori
C	Bruni

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori



Join

- ▶ **join vuoto**, nessuna tupla degli operandi è combinabile
 - ▶ ciascuna delle tuple di un operando è combinabile con tutte le tuple dell'altro. In questo caso il risultato è pari al **prodotto cartesiano** degli operandi.
- 

Join Vuoto

Impiegato	Reparto
-----------	---------

Rossi	A
-------	---

Neri	B
------	---

Bianchi	B
---------	---

Reparto	Capo
---------	------

D	Mori
---	------

C	Bruni
---	-------

Impiegato	Reparto	Capo
-----------	---------	------

Join completo

Impiegato	Reparto
-----------	---------

Rossi	B
-------	---

Neri	B
------	---

Reparto	Capo
---------	------

B	Mori
---	------

B	Bruni
---	-------

Impiegato	Reparto	Capo
-----------	---------	------

Rossi	B	Mori
-------	---	------

Rossi	B	Bruni
-------	---	-------

Neri	B	Mori
------	---	------

Neri	B	Bruni
------	---	-------

Join: Difficoltà

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparto	Capo
B	Mori
C	Bruni

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori



Join Esterno

- Il join **esterno** estende, con valori nulli, le ennuple che verrebbero tagliate fuori da un join (**interno**)
- esiste in tre versioni:
 - sinistro, destro, completo



Join Esterno

- **sinistro**: mantiene tutte le ennuple del primo operando, estendendole con valori nulli, se necessario
- **destro**: ... del secondo operando ...
- **completo**: ... di entrambi gli operandi
- ...

Join Esterno

Impiegati

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparti

Reparto	Capo
B	Mori
C	Bruni

Impiegati JOIN_{LEFT} Reparti

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL

Join Esterno

Impiegati

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparti

Reparto	Capo
B	Mori
C	Bruni

Impiegati JOIN_{RIGHT} Reparti

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
NULL	C	Bruni

Join Esterno

Impiegati

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparti

Reparto	Capo
B	Mori
C	Bruni

Impiegati JOIN_{FULL} Reparti

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL
NULL	C	Bruni



Join: PRODOTTO CARTESIANO

- Un join naturale su relazioni senza attributi in comune contiene sempre un numero di ennuple pari al prodotto delle cardinalità degli operandi (le ennuple sono tutte combinabili)
- 

Join: PRODOTTO CARTESIANO

Impiegati

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparti

Codice	Capo
A	Mori
B	Bruni

Impiegati JOIN Reparti

Impiegato	Reparto	Codice	Capo
Rossi	A	A	Mori
Rossi	A	B	Bruni
Neri	B	A	Mori
Neri	B	B	Bruni
Bianchi	B	A	Mori
Bianchi	B	B	Bruni

Join e Proiezioni

Impiegato Reparto

Rossi	A
Neri	B
Bianchi	B

Reparto Capo

B	Mori
C	Bruni

Impiegato Reparto Capo

Neri	B	Mori
Bianchi	B	Mori

Impiegato Reparto

Neri	B
Bianchi	B

Reparto Capo

B	Mori
---	------

Join: prodotto cartesiano

- Il prodotto cartesiano, in pratica, ha senso (quasi) solo se seguito da selezione:

$SEL_{Condizione} (R_1 JOIN R_2)$

- L'operazione viene chiamata **theta-join** e indicata con

$R_1 JOIN_{Condizione} R_2$

Join: prodotto cartesiano

- ▶ La condizione C è spesso una congiunzione (AND) di atomi di confronto $A_1 \vartheta A_2$ dove ϑ è uno degli operatori di confronto ($=, >, <, \dots$)
- ▶ Se l'operatore di confronto nel theta-join è sempre l'uguaglianza ($=$) allora si parla di **equi-join**

Theta JOIN ed Equi JOIN

Impiegati

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparti

Codice	Capo
A	Mori
B	Bruni

Impiegati JOIN_{Reparto=Codice} Reparti

Impiegato	Reparto	Codice	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B	B	Bruni



query

- ▶ Un'interrogazione è una funzione $E(\mathbf{r})$ che applicata ad istanze $\mathbf{r}$ di una base di dati produce una relazione su un dato insieme di attributi X .
- ▶ Le interrogazioni su uno schema di base di dati $\mathbf{R}$ in algebra relazionale sono espressioni i cui atomi (le variabili) sono relazioni in $\mathbf{R}$.

Query

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione

Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Query

Impiegati	Matricola	Nome	Età	Stipendio
	7309	Rossi	34	45
	5998	Bianchi	37	38
	9553	Neri	42	35
	5698	Bruni	43	42
	4076	Mori	45	50
	8123	Lupi	46	60

Supervisione	Impiegato	Capo
	7309	5698
	5998	5698
	9553	4076
	5698	4076
	4076	8123

Trovare matricola, nome, età e stipendio degli impiegati che guadagnano più di 40

SEL_{Stipendio>40}(Impiegati)

Query

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare matricola, nome, età e stipendio degli impiegati che guadagnano più di 40

`SELStipendio>40(Impiegati)`

Query

Impiegati	Matricola	Nome	Età	Stipendio
	7309	Rossi	34	45
	5998	Bianchi	37	38
	9553	Neri	42	35
	5698	Bruni	43	42
	4076	Mori	45	50
	8123	Lupi	46	60

Supervisione	Impiegato	Capo
	7309	5698
	5998	5698
	9553	4076
	5698	4076
	4076	8123

Trovare matricola, nome ed età degli impiegati che guadagnano più di 40

Query

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare matricola, nome ed età degli impiegati che guadagnano più di 40

SEL_{Stipendio>40}(Impiegati)

Query

Matricola	Nome	Età
7309	Rossi	34
5698	Bruni	43
4076	Mori	45
8123	Lupi	46

Trovare matricola, nome ed età degli impiegati che guadagnano più di 40

PROJ_{Matricola,}
Nome, Età (SEL_{Stipendio>40}(Impiegati))

Query

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione

Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Trovare le matricole dei capi degli impiegati che guadagnano più di 40

Query

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare le matricole dei capi degli impiegati che guadagnano più di 40

`SELStipendio>40(Impiegati)`

Query

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione

Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Trovare le matricole dei capi degli impiegati che guadagnano più di 40

Supervisione JOIN_{Impiegato=Matricola} (
SEL_{Stipendio>40}(Impiegati))

Query

Capo	Matricola	Nome	Età	Stipendio
5698	7309	Rossi	34	45
4076	5698	Bruni	43	42
8123	4076	Mori	45	50
NULL	8123	Lupi	46	60

Trovare le matricole dei capi degli impiegati che guadagnano più di 40

Supervisione JOIN_{Impiegato=Matricola} (
SEL_{Stipendio>40}(Impiegati))



Query

Capo
5698
4076
8123
NULL

Trovare le matricole dei capi degli impiegati che guadagnano più di 40

```
PROJCapo (Supervisione  
JOINImpiegato=Matricola (SELStipendio>40 (Im  
piegati)))
```


Query

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione

Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Trovare le matricole dei capi degli impiegati che guadagnano più di 40

PROJ_{Capo} (Supervisione
JOIN_{Impiegato=Matricola} (SEL_{Stipendio>40} (I
mpiegati)))

Query

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione

Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

Query

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

`SELStipendio>40(Impiegati)`

Query

Capo	Matricola	Nome	Età	Stipendio
5698	7309	Rossi	34	45
4076	5698	Bruni	43	42
8123	4076	Mori	45	50
NULL	8123	Lupi	46	60

Supervisione

Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

Supervisione JOIN_{Impiegato=Matricola} (SEL_{Stipendio>40}(Impiegati))

Query

Capo	Matricola	Nome	Età	Stipendio
5698	7309	Rossi	34	45
4076	5698	Bruni	43	42
8123	4076	Mori	45	50
NULL	8123	Lupi	46	60

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

Supervisione JOIN_{Impiegato=Matricola} (
SEL_{Stipendio>40}(Impiegati))

Query

Capo	Matricola	Nome	Età	Stipendio
5698	7309	Rossi	34	45
4076	5698	Bruni	43	42
8123	4076	Mori	45	50
NULL	8123	Lupi	46	60

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

$\text{PROJ}_{\text{Capo}}(\text{Supervisione JOIN}$
 $\text{Impiegato=Matricola (SEL}_{\text{Stipendio}}$
 $>40(\text{Impiegati})))$

Query

Capo
5698
4076
8123
NULL

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

$\text{PROJ}_{\text{Capo}}(\text{Supervisione JOIN}$
 $\text{Impiegato=Matricola (SEL}_{\text{Stipendio}}$
 $>40(\text{Impiegati})))$

Query

Capo

5698

4076

8123

NULL

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

Impiegati JOIN_{Matricola=Capo}
PROJ_{Capo}(Supervisione JOI
N_{Impiegato=Matricola}(SEL_{Stipend}
io>40(Impiegati)))

Query

Matricola	Nome	Età	Stipendio
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

Impiegati JOIN_{Matricola=Capo}
PROJ_{Capo}(Supervisione JOI
N_{Impiegato=Matricola} (SEL_{Stipend}
io>40(Impiegati)))

Query

Nome	Stipendio
Bruni	42
Mori	50
Lupi	60

Trovare nome e stipendio
dei capi degli impiegati
che guadagnano più di 40

```
PROJNome,Stipendio ( Impiegati  
JOINMatricola=Capo  
PROJCapo(Supervisione JOI  
NImpiegato=Matricola (SELStipend  
io>40(Impiegati))))
```

Query

Impiegati

Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione

Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Trovare gli impiegati che guadagnano più del proprio capo, mostrando matricola, nome e stipendio dell'impiegato e del capo

Query

Capo	Matricola	Nome	Età	Stipendio
5698	7309	Rossi	34	45
5698	5998	Bianchi	37	38
4076	9553	Neri	42	35
4076	5698	Bruni	43	42
8123	4076	Mori	45	50
NULL	8123	Lupi	46	60

Trovare gli impiegati che guadagnano più del proprio capo, mostrando matricola, nome e stipendio dell'impiegato e del capo

Supervisione JOIN_{Impiegato=M}
a
tricola Impiegati

Query

Capo	Matricola	Nome	Età	Stipendio
5698	7309	Rossi	34	45
5698	5998	Bianchi	37	38
4076	9553	Neri	42	35
4076	5698	Bruni	43	42
8123	4076	Mori	45	50
NULL	8123	Lupi	46	60

MatrC	NomeC	EtàC	StipendioC
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare gli impiegati che guadagnano più del proprio capo, mostrando matricola, nome e stipendio dell'impiegato e del capo

REN_{MatrC, NomeC, StipC, EtàC} ← Matr, Nome, Stip, Età (Impiegati)

Query

MatrC	NomeC	EtàC	StipendioC	Matricola	Nome	Età	Stipendio
5698	Bruni	43	42	7309	Rossi	34	45
5698	Bruni	43	42	5998	Bianchi	37	38
4076	Mori	45	50	9553	Neri	42	35
4076	Mori	45	50	5698	Bruni	43	42
8123	Lupi	46	60	4076	Mori	45	50
NULL	NULL	NULL	NULL	8123	Lupi	46	60

Trovare gli impiegati che guadagnano più del proprio capo, mostrando matricola, nome e stipendio dell'impiegato e del capo

```
RENMatrC, NomeC, StipC, EtàC □ Ma  
tr, Nome, Stip, Età (Impiegati)  
JOINMatrC=Capo  
(Supervisione JOINImpie  
gato=Matricola Impiegati)
```

Query

MatrC	NomeC	EtàC	StipendioC	Matricola	Nome	Età	Stipendio
5698	Bruni	43	42	7309	Rossi	34	45

Trovare gli impiegati che guadagnano più del proprio capo, mostrando matricola, nome e stipendio dell'impiegato e del capo

```
SELStipendio>StipC (  
RENMatrC, NomeC, StipC, EtàC □ Matr, Nome, Stip, Età (I  
mpiegati)  
JOINMatrC=Capo  
(Supervisione JOINImpiegato=Matricola  
a Impiegati))
```


Query

MatrC	NomeC	StipendioC	Matricola	Nome	Stipendio
5698	Bruni	42	7309	Rossi	45

Trovare gli impiegati che guadagnano più del proprio capo, mostrando matricola, nome e stipendio dell'impiegato e del capo

```
PROJMatr, Nome, Stip, MatrC, NomeC, StipC  
  (SELStipendio > StipC  
RENMatrC, NomeC, StipC, EtàC ⋈Matr, Nome, Stip, Età  
  (Impiegati)  
  JOINMatrC=Capo  
(Supervisione JOINImpiegato=M  
  atricola Impiegati)))
```




VISTE

- Rappresentazioni diverse per gli stessi dati (schema esterno)
 - Relazioni derivate:
 - relazioni il cui contenuto è funzione del contenuto di altre relazioni (definito per mezzo di interrogazioni)
 - Relazioni di base: contenuto autonomo
- 



VISTE

Vantaggi

- ▶ Permettono di mostrare a un utente le sole componenti della base di dati che interessano
- ▶ Espressioni molto complesse possono essere definite come viste
- ▶ Sicurezza: è possibile definire diritti di accesso anche relativamente ad una vista (e quindi ad una particolare porzione della base di dati)
- ▶ In caso di ristrutturazione della base di dati, le “vecchie” relazioni possono essere di nuovo ricavate mediante viste, consentendo l'uso di applicazioni che fanno riferimento al vecchio schema

VISTE

Afferenza

Impiegato Reparto

Rossi	A
Neri	B
Bianchi	B

Direzione

Reparto	Capo
---------	------

A	Mori
B	Bruni

Supervisione =

PROJ_{Impiegato, Capo} (Afferenza JOIN
Direzione)



VISTE Motivazioni

- Schema esterno: ogni utente vede solo
 - ciò che gli interessa e nel modo in cui gli interessa, senza essere distratto dal resto
 - ciò che e' autorizzato a vedere (autorizzazioni)
- Strumento di programmazione:
 - si può semplificare la scrittura di interrogazioni: espressioni complesse e sottoespressioni ripetute
- Utilizzo di programmi esistenti su schemi ristrutturati

Invece:

- L'utilizzo di viste non influisce sull'efficienza delle interrogazioni



Viste come strumento di programmazione

- Trovare gli impiegati che hanno lo stesso capo di Rossi
- Senza vista:

```
PROJ Impiegato ((Afferenza JOIN Direzione) JOIN  
                REN ImpR,RepR ← Imp,Reparto (  
                SEL Impiegato='Rossi' (Afferenza JOIN Direzione))))
```

- Con la vista:

```
PROJ Impiegato (Supervisione JOIN  
                REN ImpR,RepR ← Imp,Reparto (  
                SEL Impiegato='Rossi' (Supervisione))))
```



Tool per algebra relazionale

RelaX

➤ <https://dbis-uibk.github.io/relax/calc/local/uibk/local/0>

The screenshot displays the RelaX web interface. On the left, a sidebar titled 'Select DB (UIBK - R, S, T)' lists three databases: R (with attributes a: number, b: string, c: string), S (with attributes b: string, d: number), and T (with attributes b: string, d: number). The main area has tabs for 'Relational Algebra', 'SQL', and 'Group Editor'. The 'Relational Algebra' tab is active, showing a query editor with the expression $\pi_{a,b,c}(R)$. Below the editor is a blue 'execute query' button. To the right of the button are 'download' and 'history' links. At the bottom, a query execution plan is shown, consisting of a box for $\pi_{a,b,c}$ (5 rows) connected to a box for R (5 rows). The overall expression $\pi_{a,b,c}(R)$ is displayed at the bottom of the interface.